

SOPHI: An Open End-to-End Camera–LiDAR Planning Stack

with Sparse-Convolution Inference for NAVSIM

Atharv Sharma

ATHARV228.AS@GMAIL.COM

Abstract

We release **SOPHI** (*Sparse-conv Offline Perception with Hydra-distillation Inference*), an open Apache-2.0 software stack for training, evaluating, and deploying an end-to-end autonomous driving planner on NAVSIM. A fused camera–LiDAR bird’s-eye-view (BEV) feature $\mathbf{F}_{\text{env}}$ is shared by a Hydra-MDP-style trajectory decoder and auxiliary 3D detection and BEV segmentation heads. NVIDIA’s CUDA-BEVFusion reference provides a high-performance detection runtime but ships a closed LiDAR sparse-convolution (SCN) binary and omits segmentation and planning. SOPHI replaces that closed SCN path with open `spconv 2.x`, exports the full multi-head graph to ONNX/TensorRT, and documents a reproducible train→evaluate→export→C++ deploy workflow so others can build their own models. On an RTX 3060 we measure ~40 ms LiDAR SCN latency (~391 ms end-to-end C++ FP16). On the NAVSIM `navtest` split (12,149 scenarios) the released checkpoint scores PDM 0.7925 (NC 0.982, DAC 0.975, TTC 0.981). Python and C++ trajectories agree to within 1.7 mm max absolute error (cosine similarity 1.000).

Keywords: open source software, autonomous driving, bird’s-eye view, sparse convolution, TensorRT, NAVSIM

1 Introduction

Learned planners such as Hydra-MDP Chen et al. (2024) and GTRS NVIDIA Corporation (2024) score a trajectory vocabulary from a dense BEV environment encoding. BEVFusion Liu et al. (2023) and NVIDIA’s CUDA-BEVFusion NVIDIA Corporation (2022) supply a strong camera–LiDAR fusion template for detection, but the reference deployment (i) depends on a non-redistributable `libspconv` for the LiDAR SCN, (ii) exports detection only, and (iii) does not document a closed loop from NAVSIM Dauner et al. (2024) training through TensorRT planning.

SOPHI fills that gap as *software*, not as a new planning algorithm. We reuse the public BEVFusion backbone and a Hydra-style distillation decoder, and contribute: (1) an open SCN ONNX executor on `spconv 2.x` Yan (2022); (2) ONNX/TensorRT export and a C++ runtime for planning, detection, and BEV segmentation on a shared $\mathbf{F}_{\text{env}}$; (3) a single-environment training and PDM evaluation recipe; (4) validated `navtest` scores and Python↔C++ parity. The goal is that a researcher can download the stack, retrain or swap heads, and redeploy without the closed SCN binary.

Related software. We build on BEVFusion Liu et al. (2023), lift-splat-shoot Phillion and Fidler (2020), SECOND-style sparse ops Yan et al. (2018), open `spconv` Yan (2022), Hydra-MDP/GTRS Chen et al. (2024); NVIDIA Corporation (2024), and NAVSIM/PDM Dauner et al. (2024, 2023). Our contribution is the open integration and deployment path.

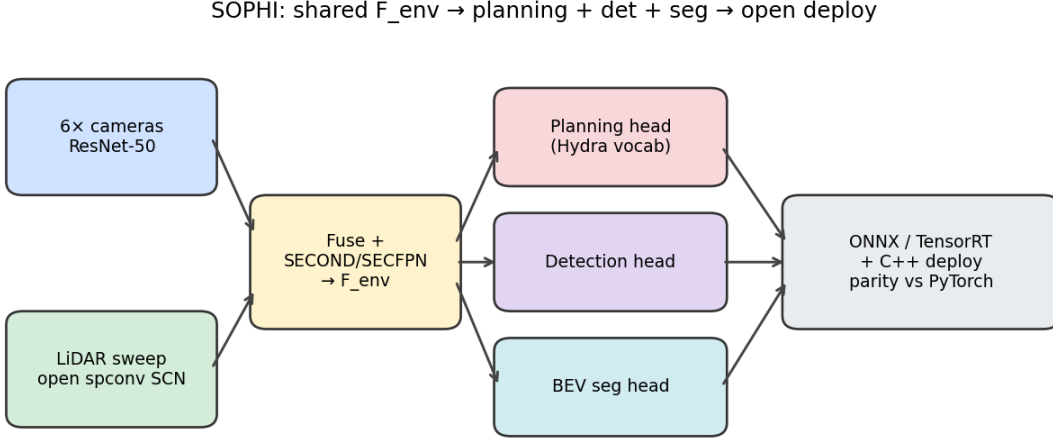


Figure 1: SOPHI layout. Camera (ResNet-50/LSS) and open-spconv LiDAR SCN fuse into F_{env} , consumed by planning, detection, and BEV-segmentation heads. TensorRT runs dense graphs; SCN runs via the open parser.

2 Software Overview

Repository layout. The release (Apache-2.0) is at <https://github.com/atharvsharma1998/hydra-mdp> (tag v0.1.0-mloss): `navsim/agents/gtrs_bevfusion/` (model/losses), `scripts/training/train`, `scripts/export/export_gtrs_bevfusion_onnx.py`, and `deploy/` (C++/TensorRT NVIDIA Corporation (2023) + parity). `REPRODUCIBILITY.md` lists copy-paste commands for each step below.

Architecture. Camera images and a LiDAR sweep produce $F_{\text{env}} \in \mathbb{R}^{512 \times H \times W}$ via LSS view transform Phillion and Fidler (2020), voxel SCN, convolutional fusion, and a SECOND/SECFPN decoder (Figure 1). F_{env} is tokenized and fed to a Transformer trajectory scorer with a frozen vocabulary (up to $K=8192$), trained by soft imitation and per-proposal PDM sub-score distillation Dauner et al. (2023). Auxiliary CenterPoint-style detection and a multiclass BEV segmentation head (class-frequency CE + Lovász-softmax Berman et al. (2018)) supervise the same feature for representation shaping and debugging; they do not gate the plan.

Open SCN runtime. The LiDAR ONNX graph (~ 53 nodes) is executed by `ONNXLiDARModel`: FP16 KRSC weights on GPU, topological spconv 2.x ops (`kMaskImplicitGemm` with tuner cache, SubM pair-table reuse, grow-only scratch buffers). This replaces NVIDIA’s closed `libspconv` while preserving the exported graph interface. Dense graphs (camera backbone/vtransform, fuser, planning/det/seg heads) are built with TensorRT 8.6 FP16 engines; the camera ResNet uses FP32 compute with FP16 I/O to avoid overflow.

3 Reproducibility Workflow

Anyone adapting SOPHI follows five steps (details in `REPRODUCIBILITY.md`): (1) Environment — Python 3.8, PyTorch 1.10+cu113, `mmcv-full` 1.4, OpenMMLab `mmdet3d` (CUDA-BEVFusion fork), `spconv` 2.x, TensorRT 8.6; set `BEVFUSION_ROOT`, `NUPLAN_MAPS_ROOT`, `PYTHONPATH`. (2) Data — download NAVSIM logs/sensors/maps; run `run_metric_caching.py` with a process pool. (3) Train — `train_gtrs_bevfusion.py` on `navtrain` (raw model `state_dict`; agent loader strips optional `agent./model.` prefixes). (4) Evaluate — Hydra agent `gtrs_bevfusion_agent` via `run_pdm_score.py` on `navtest` with `worker.use_process_pool=True` (~ 10 workers on an 80 GB A100). (5) Deploy — export ONNX (`--lidar-scnn`), delete stale `.plan` files, `build_gtrs_engines.sh fp16`, run `gtrs_bevfusion`, then `compare_cpp_parity.py`. An early rasterize-then-plan prototype failed when map thresholds discarded latent detail; SOPHI therefore plans from tokenized $\mathbf{F}_{\text{env}}$. Map training uses frequency-weighted CE + Lovász so thin classes are not dominated by road/background.

4 Experimental Validation

Latency (RTX 3060 12 GB, FP16). Table 1 reports steady-state C++ timings after open-SCN optimization (20-iteration mean; first-frame tuner warmup excluded). End-to-end wall latency is ~ 404 ms (~ 2.5 FPS) on this consumer GPU.

Table 1: C++ FP16 module latency on RTX 3060 (ms, mean).

Module	ms
LiDAR (range-crop + open SCN)	37.8
Camera (LSS)	169.8
Fuser ($\mathbf{F}_{\text{env}}$)	1.6
Heads (plan + det + seg)	182.2
Total (device)	391.3

NAVSIM navtest PDM. On the official single-stage `navtest` split (12,149 scenarios; metric cache precomputed), the released checkpoint `gtrs_bevfusion_navtrain_v1.best.pth` scores **PDM** 0.7925. Stage-one sub-metrics: no-at-fault collision 0.982, drivable-area compliance 0.975, driving-direction 0.996, traffic-light 0.998, ego progress 0.854, time-to-collision 0.981, lane keeping 0.970, history comfort 0.964. Safety/compliance are near-saturated; ego progress is the remaining soft spot (conservative plans). Two-stage reactive EPDMS columns are absent because `navtest` does not ship synthetic reactive frames.

Python $\leftrightarrow$ C++ parity. On token `8bc34517e08758ff`, stage-wise cosine similarities are ≥ 0.993 (Table 2). The deployed trajectory matches PyTorch to 1.7×10^{-3} m max absolute error.

Table 2: Python vs. C++/TensorRT parity (cosine / $\max|\Delta|$).

Stage	Cosine	Max $ \Delta $
lidar_bev / cam_bev / $\mathbf{F}_{\text{env}}$	0.993 / 1.000 / 0.997	2.35 / 0.10 / 0.73
scores / trajectory	0.9995 / 1.000	1.57 / 0.0017
agent states / class logits	1.000 / 1.000	0.066 / 0.066
bev_semantic_logits	0.9998	1.27

5 Licensing and Limitations

What is open. Training, export, deploy sources, and documentation are Apache-2.0. Open `spconv` executes the LiDAR SCN. We build on NVIDIA’s Apache-2.0 CUDA-BEVFusion *template* and replace the non-redistributable SCN binary with open inference. We do *not* claim a fully proprietary-free stack: CUDA and TensorRT remain required for the documented C++ path (widely used in the community; justified analogously to other MLOSS GPU libraries). NAVSIM/nuPlan data must be obtained under their own licenses.

Scope. SOPHI reuses Hydra-MDP/GTRS scoring and BEVFusion fusion; novelty is the open SCN path, multi-head deploy graph, and end-to-end recipe. PDM 0.7925 is competitive but not state-of-the-art (e.g., Transfuser is higher on navtest). Further work includes int8 engines, ego-progress calibration, and two-stage navhard EPDMS.

6 Conclusion

SOPHI is an open, documented camera–LiDAR–planning stack for NAVSIM: train multi-task $\mathbf{F}_{\text{env}}$ models, score them with official PDM evaluation, and deploy with open SCN + TensorRT while matching Python trajectories at millimeter scale. We hope it lowers the barrier for researchers who want to modify sparse LiDAR backbones and ship full perception-to-planning runtimes without closed SCN libraries.

Acknowledgments and Disclosure of Funding

No external funding was received. Source code and the submission archive are released under Apache-2.0 at <https://github.com/atharvsharma1998/hydra-mdp> (tag v0.1.0-mloss). Companion instructions: `jmlr/REPRODUCIBILITY.md`.

References

- Maxim Berman, Amal Rannen Triki, and Matthew B Blaschko. The lovász-softmax loss: A tractable surrogate for the optimization of the intersection-over-union measure in neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- Shoufa Chen, Shaoshuai Jiang, Hongyang Li, Yue Jia, Hongsheng Gao, Junchi Yan, and Hongyang Li. Hydra-MDP: End-to-end multimodal planning with hydra distillation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

- Daniel Dauner, Marcel Hallgarten, Andreas Geiger, and Kashyap Chitta. Parting with misconceptions about learning-based vehicle motion planning. In *Conference on Robot Learning (CoRL)*, 2023.
- Daniel Dauner, Marcel Hallgarten, Tianyu Li, Xinshuo Weng, Zhiyu Huang, Zetong Yang, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.
- Zhijian Liu, Haotian Tang, Alexander Amini, Jing Yang, Hanjie Mao, Daniela Rus, and Song Han. Bevfusion: Multi-task multi-sensor fusion with unified bird’s-eye view representation. In *International Conference on Robotics and Automation (ICRA)*, 2023.
- NVIDIA Corporation. CUDA-BEV Fusion: High-performance 3D perception for autonomous driving. https://github.com/NVIDIA-AI-IOT/Lidar_AI_Solution, 2022. Accessed 2026-06-13.
- NVIDIA Corporation. TensorRT: High-performance deep learning inference. <https://developer.nvidia.com/tensorrt>, 2023.
- NVIDIA Corporation. GTRS: Generalized trajectory scoring for autonomous driving. <https://github.com/NVIDIA/DRIVE-Lab/GTRS>, 2024. Accessed 2026-06-13.
- Jonah Philion and Sanja Fidler. Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3D. In *European Conference on Computer Vision (ECCV)*, 2020.
- Yan Yan. Spconv: Spatially sparse convolution library. *GitHub repository*, 2022. <https://github.com/traveller59/spconv>.
- Yan Yan, Yuxin Mao, and Bo Li. SECOND: Sparsely embedded convolutional detection. In *Sensors*, 2018.